Lecture 12

Metaprogramming

Example Code and Exercises

`https://github.com/eecs498-software-design/lecture`

Metaprogramming - General Concepts

In metaprogramming, a program treats itself as data.

- Introspection

  The fundamental capability of a program to inspect and reason about itself.

- Intercession

  Augmenting or modifying the behavior of a program.

- Generation

  Creating new code or behaviors based on an existing program.

Metaprogramming – When/How

- Languages have **different approaches** to metaprogramming.
- The effects may be handled at **compile-time** or **runtime**.

Examples:

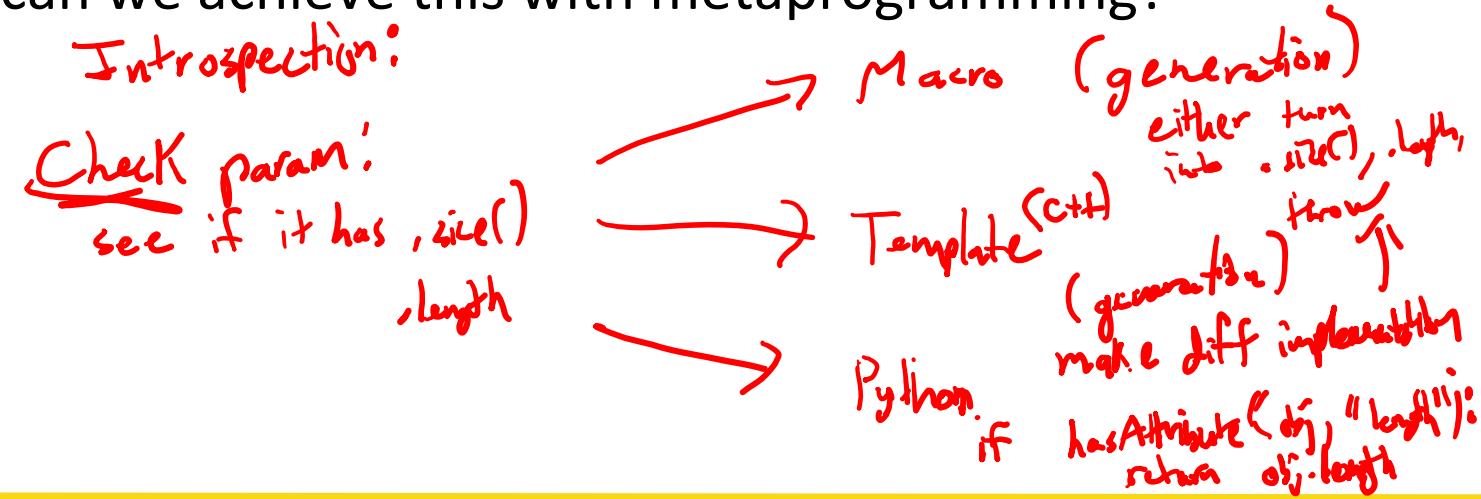
- C++ Template Metaprogramming with *SFINAE* or concepts.
  Use templates to conditionally generate code at compile-time.
- Example: Typescript Type-Level Metaprogramming
  Use mapped/conditional types to generate compile-time types matching underlying dynamic introspection/intercession.

Example

get_size(map)
get_size(arr)
get_size(2)

Goal: Create a single function that retrieves either `.size()` or `.length` on any object. If neither is present, throws an exception.

How can we achieve this with metaprogramming?



C++ Example, SFINAE

Goal: Create a single function that retrieves either `.size()` or `.length` on any object. If neither is present, throws an exception.

- **S**ubstitution **F**ailure **I**s **N**ot **A**n **E**rror

```cpp
// Only viable if T has a .size() member function
template <typename T>
auto get_size(const T& obj) -> decltype(obj.size()) {
  return obj.size();
}

// Only viable if T has a .length member variable
template <typename T>
auto get_size(const T& obj) -> decltype(obj.length) {
  return obj.length;
}

// Fallback for types without .size() or .length
int get_size(...) {
  throw std::runtime_error("No size available");
}
```

```cpp
int main() {
  vector<int> vec = {1, 2, 3};
  int x = 42;

  Buffer<int, 4> buf = {{1, 2, 3, 4}};

  try {
    // returns int from .size()
    cout << get_size(vec);

    // returns int from .size()
    cout << get_size(buf);

    // returns int, wait jk it throws
    cout << get_size(x);

  } catch (const runtime_error& e) {
    cout << "Caught: " << e.what();
  }
}
```

TS Example

Goal: Create a single function that retrieves either `.size()` or `.length` on any object. If neither is present, throws an exception.

- Dynamically introspect for property.

```ts
function getSize(x: unknown): number {
  if (typeof x === 'object' && x !== null) {
    if ('size' in x && typeof x.size === 'number') {
        return x.size;
    }
    if ('length' in x && typeof x.length === 'number') {
      return x.length;
    }
  }
  throw new Error("No size available");
}
```

```ts
const map = new Map<string, number>(...);
const x = 42;
const arr = [1, 2, 3, 4];

try {
  // returns number from .size property
  console.log(getSize(map));

  // returns number from .length property
  console.log(getSize(arr));

  // returns number, wait jk it throws
  console.log(getSize(x));
} catch (e) {
  if (e instanceof Error) {
    console.log("Caught: ", e.message);
  } else { throw e; }
}
```

Improved TS Example

Goal: Create a single function that retrieves either `.size()` or `.length` on any object. If neither is present, throws an exception.

- Use Type-Level Metaprogramming to generate the appropriate types.

```ts
// Type guard: checks if object has a .size property
function hasSize(obj: unknown): obj is { size: number } {
  return (
    typeof obj === "object" && obj !== null &&
    "size" in obj && typeof (obj as any).size === "number"
  );
}

// Type guard: checks if object has a .length property
function hasLength(obj: unknown): obj is { length: number } {
  // Defined similarly
}

function getSize(obj: unknown): number {
  if (hasSize(obj)) { return obj.size; }
  if (hasLength(obj)) { return obj.length; }
  throw new Error("No size available");
}
```

```ts
const map = new Map<string, number>(...);
const x = 42;
const arr = [1, 2, 3, 4];

try {
  // returns number from .size property
  console.log(getSize(map));

  // returns number from .length property
  console.log(getSize(arr));

  // returns number, wait jk it throws
  console.log(getSize(x));
} catch (e) {
  if (e instanceof Error) {
    console.log("Caught: ", e.message);
  } else { throw e; }
}
```

Practical Metaprogramming

```
export interface SpellingBeeObserver extends PuzzleObserver {
  onWordGuessed?(word: string, result: GuessResult): void;
  onShuffled?(order: string[]): void;
  onHintGiven?(hint: string): void;
}
```

- Goal: Create a reusable class that can act as a component of an observable entity.
- The `ObserverT` parameter specifies an interface with functions for each event.
- The emit function is generic and "works" for any event K.
  For example:

✅ `emit("onShuffled", ["x","y","z"])`

❌ `emit("onHintGiven") // missing arg`

❌ `emit("onShuff") // bad event name`

```
export class Observable<ObserverT extends object> {
  private observers = new Set<ObserverT>();

  registerObserver(observer: ObserverT): () => void {
    this.observers.add(observer);
    return () => this.unregisterObserver(observer);
  }

  // Emit an event to all observers that implement the callback.
  // Type-safe: event name and args are checked against ObserverT.
  emit<K extends keyof ObserverT>(
    event: K,

    ...args: NonNullable<ObserverT[K]> extends ((...args: infer A) => void)
             ? A
             : never

  ): void {
    [...this.observers].forEach(observer => {
      const fn = observer[event];
      if (fn) { (fn as any).apply(observer, args); }
    });
  }
}
```

1. Ensure K is an actual property key from ObserverT. e.g. "onHintGiven"

2. `ObserverT[K]` gives the type of the event handler for K. e.g. `(hint: string)=>void`.

3. args has a conditional type, based on the `extends` clause (above) to check if the handler is actually a function. The two branches are:
`? A` Handler is a function, so infer its args A (above). `emit` takes same args.
`: never` Handler is not a function, use never to tank compilation of `emit`.

4. Finally, the implementation just grabs the handler for this event, checks if it's not null, and then just "trusts" with an any cast. In Typescript, metaprogramming is often done at the type level, then once verified you just switch to Javascript which lets you do whatever.

NOTE: The in-class demo examples of metaprogramming in W26 were not great. I finally found a nicer example during P4 prep with the Observable class, so I've included it here.